Taller: API REST con Node.js, Express y JWT

Nombre: Jenny Vásquez

Entidad seleccionada: libros (id, titulo, autor, anio, genero)

Base de datos: SQLite (archivo data/libros.db) · **Pruebas:** peticiones HTTP a http://localhost:3000 con token Bearer

---

Evidencias de las operaciones y de la base de datos

Cada bloque muestra la petición enviada, la respuesta real de la API y, cuando corresponde, la consulta directa a la base de datos. Los tokens aparecen recortados.

1. Login con credenciales válidas

Petición

```
POST /auth/login

{
  "username": "admin",
  "password": "********"
}
```

Respuesta: HTTP 200

```
{
  "token": "eyJhbGciOiJIUzI1Ni…[recortado]",
  "tipo": "Bearer",
  "expira_en": "15m"
}
```

2. Login con credenciales incorrectas

Petición

```
POST /auth/login

{
  "username": "admin",
  "password": "********"
}
```

Respuesta: HTTP 401

```
{
  "error": "Credenciales incorrectas"
}
```

3. Crear libro con JWT

Petición

```
POST /libros
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]

{
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967,
  "genero": "Novela"
}
```

Respuesta: HTTP 201

```
{
  "id": 1,
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967,
  "genero": "Novela",
  "creado_en": "2026-11-09 04:19:31"
}
```

Consulta directa a la base de datos

```
SELECT * FROM libros WHERE id = 1

[
  {
    "id": 1,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "anio": 1967,
    "genero": "Novela",
    "creado_en": "2026-11-09 04:19:31"
  }
]
```

4. Consultar la lista

Petición

```
GET /libros
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]
```

Respuesta: HTTP 200

```
[
  {
    "id": 1,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "anio": 1967,
    "genero": "Novela",
    "creado_en": "2026-11-09 04:19:31"
  }
]
```

5. Consultar el registro por identificador

Petición

```
GET /libros/1
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]
```

Respuesta: HTTP 200

```
{
  "id": 1,
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967,
  "genero": "Novela",
  "creado_en": "2026-11-09 04:19:31"
}
```

6. Actualizar el campo "genero"

Petición

```
PUT /libros/1
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]

{
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967,
  "genero": "Realismo mágico"
}
```

Respuesta: HTTP 200

```
{
  "id": 1,
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967,
  "genero": "Realismo mágico",
  "creado_en": "2026-11-09 04:19:31"
}
```

Consulta directa a la base de datos

```
SELECT * FROM libros WHERE id = 1

[
  {
    "id": 1,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "anio": 1967,
    "genero": "Realismo mágico",
    "creado_en": "2026-11-09 04:19:31"
  }
]
```

7. Tras reiniciar Node.js: consultar de nuevo el registro

Petición

```
GET /libros/1
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]
```

Respuesta: HTTP 200

```
{
  "id": 1,
  "titulo": "Cien años de soledad",
  "autor": "Gabriel García Márquez",
  "anio": 1967,
  "genero": "Realismo mágico",
  "creado_en": "2026-11-09 04:19:31"
}
```

Consulta directa a la base de datos

```
SELECT * FROM libros WHERE id = 1

[
  {
    "id": 1,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "anio": 1967,
    "genero": "Realismo mágico",
    "creado_en": "2026-11-09 04:19:31"
  }
]
```

8. GET /libros sin token

Petición

```
GET /libros
```

Respuesta: HTTP 401

```
{
  "error": "Token requerido (Authorization: Bearer <token>)"
}
```

9. POST /libros sin token

Petición

```
POST /libros

{
  "titulo": "X",
  "autor": "Y"
}
```

Respuesta: HTTP 401

```
{
  "error": "Token requerido (Authorization: Bearer <token>)"
}
```

10. DELETE /libros/:id con token inválido

Petición

```
DELETE /libros/1
Authorization: Bearer token.invalido.123…[recortado]
```

Respuesta: HTTP 401

```
{
  "error": "Token inválido"
}
```

11. PUT /libros/:id con token vencido

Petición

```
PUT /libros/1
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]

{
  "titulo": "Hack",
  "autor": "Hack"
}
```

Respuesta: HTTP 401

```
{
  "error": "Token vencido"
}
```

Consulta directa a la base de datos

```
SELECT * FROM libros ORDER BY id

[
  {
    "id": 1,
    "titulo": "Cien años de soledad",
    "autor": "Gabriel García Márquez",
    "anio": 1967,
    "genero": "Realismo mágico",
    "creado_en": "2026-11-09 04:19:31"
  }
]
```

Verificación: peticiones sin token, con token inválido y con token vencido

Se comparó el contenido completo de la tabla libros antes y después de las peticiones 8 a 11: **sin cambios**.

12. Crear sin campo obligatorio (titulo)

Petición

```
POST /libros
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]

{
  "autor": "Sin título"
}
```

Respuesta: HTTP 400

```
{
  "errores": [
    "titulo es obligatorio (texto)"
  ]
}
```

Consulta directa a la base de datos

```
SELECT COUNT(*) AS total FROM libros

[
  {
    "total": 1
  }
]
```

13. Eliminar el registro

Petición

```
DELETE /libros/1
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]
```

Respuesta: HTTP 200

```
{
  "mensaje": "Libro 1 eliminado"
}
```

Consulta directa a la base de datos

```
SELECT * FROM libros WHERE id = 1

(sin filas)
```

14. Consultar el registro eliminado

Petición

```
GET /libros/1
Authorization: Bearer eyJhbGciOiJIUzI1Ni…[recortado]
```

Respuesta: HTTP 404

```
{
  "error": "No existe el libro con id 1"
}
```

Resumen de resultados

```
HTTP 200  1. Login con credenciales válidas
HTTP 401  2. Login con credenciales incorrectas
HTTP 201  3. Crear libro con JWT
HTTP 200  4. Consultar la lista
HTTP 200  5. Consultar el registro por identificador
HTTP 200  6. Actualizar el campo "genero"
HTTP 200  7. Tras reiniciar Node.js: consultar de nuevo el registro
HTTP 401  8. GET /libros sin token
HTTP 401  9. POST /libros sin token
HTTP 401  10. DELETE /libros/:id con token inválido
HTTP 401  11. PUT /libros/:id con token vencido
HTTP 400  12. Crear sin campo obligatorio (titulo)
HTTP 200  13. Eliminar el registro
HTTP 404  14. Consultar el registro eliminado
```